Q1:

a. Fourier transform is better than geometric operations when the problem we want to solve is hard in the spatial domain, but it can be solved (relatively easier) in the frequency domain.

   I'd say that in most cases Fourier transform is better since we can create a lot of frequency-related problems which are pretty hard to solve in the spatial domain, but I argue that a most of problems which is represented in the spatial domain can be solved using both geometric operation and Fourier as well, which makes Fourier preferable.

   Not to forget that geometric operations can be expensive (even though they managed to reduce the complexity but still much slower than Fourier). Depending on the interpolation method, artifacts like aliasing or blurring may appear, which doesn't happen using fourier.

   In general I think Fourier can more practical solution.

b. Given that $\forall u, v, G(u,v) = F(u,v)$ where F, G are Fourier transforms for f, g respectively. We want to prove that $\forall x, y, g(x,y) = f(x,y)$. The proof is derived directly from the definition:

$$g(x,y) = \sum_{u=0}^{N-1}\sum_{v=0}^{M-1} G(u,v)\cdot e^{2i\pi\left(\frac{ux}{N}+\frac{vy}{M}\right)} = \sum_{u=0}^{N-1}\sum_{v=0}^{M-1} F(u,v)\cdot e^{2i\pi\left(\frac{ux}{N}+\frac{vy}{M}\right)} = f(x,y)$$

   Where the third equality is because $G = F$.

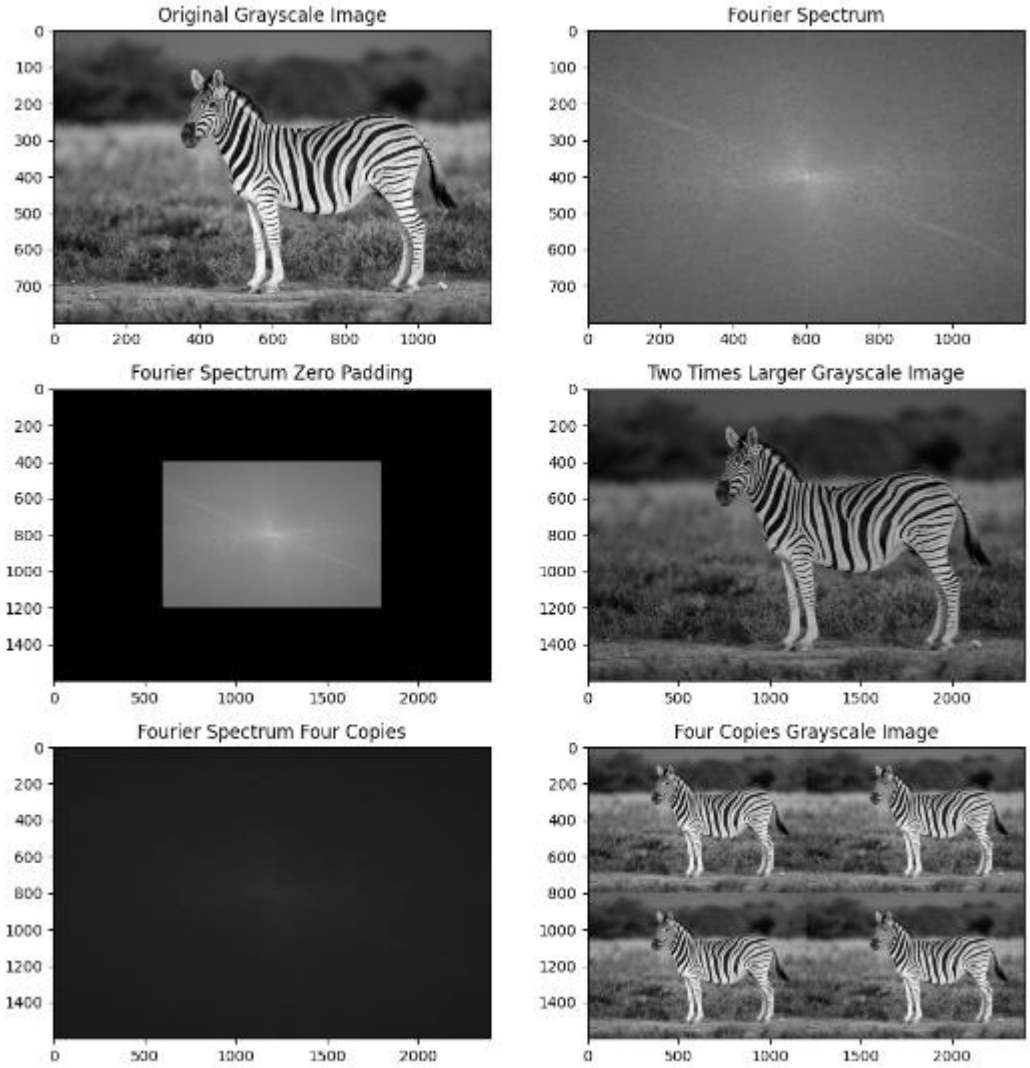
Q2:

a. This part is straight forward, we simply gray scale the image then represent it in the frequency domain.

b. We created a blank black image, then we represented the image in the frequency domain and put it in the center of the blank. Then return the image back to the spatial domain.

c. We put zero between each 2 coefficients in the frequency domain instead of surrounding the transform of the image with zeros and that is the difference between the first and second methods.

   Here's the final result:



Q3:

Baby image:

In this image, we cleaned the 3 images given individually, then we applied a perspective transformation with corners as source points and the $\{0,255\} \times \{0,255\}$ as destination points (since we want one image with size 256×256). We found the corners using this code:

```
img1_path = 'Images/baby.tif'

# Load the noisy image
image = cv2.imread(img1_path, cv2.IMREAD_GRAYSCALE)
# Plot the image
plt.imshow(image)
plt.axis('off')
plt.show()
```

The code:

```
def clean_baby(im):
    # Define destination points for perspective transformation - corners
    dest_location = np.float32([[0, 0], [0, 255], [255, 0], [255, 255]])

    # Define source points for each image - corners of each image
    img1_src = np.float32([[5, 19], [5, 131], [111, 19], [111, 131]])
    img2_src = np.float32([[181, 4], [121, 51], [249, 70], [177, 120]])
    img3_src = np.float32([[78, 163], [133, 244], [145, 117], [245, 160]])
    source_locations = [img1_src, img2_src, img3_src]

    # Transform source points to NumPy arrays
    source_locations = [np.array(pts, dtype=np.float32) for pts in source_locations]

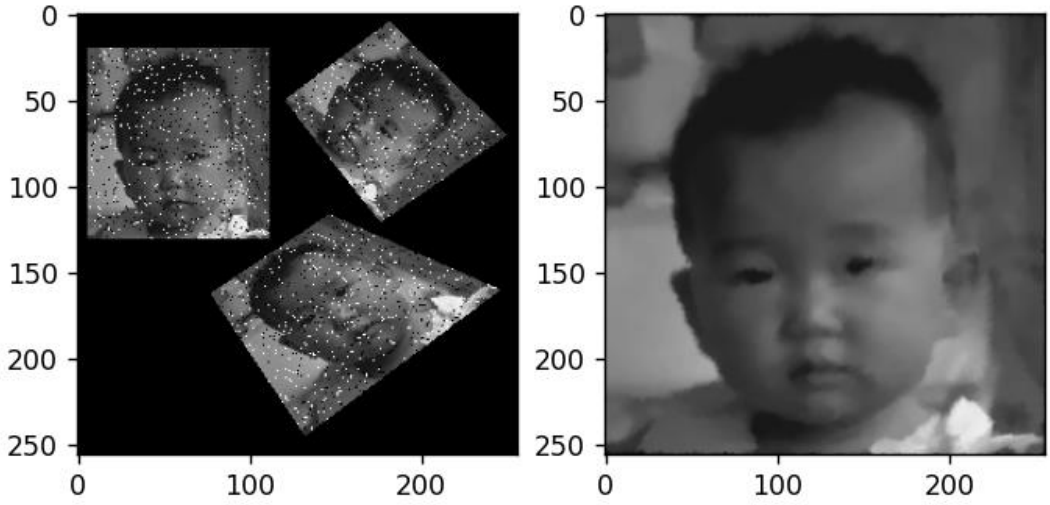
    # List to store transformed images
    transformed_images = []

    # Process each source point set
    for location in source_locations:
        # Remove noise using a median filter
        denoised_image = cv2.medianBlur(im, ksize: 3)
        # Compute the perspective transformation matrix
        transform_matrix = cv2.getPerspectiveTransform(location, dest_location)
        # Apply the perspective transformation
        transformed_image = cv2.warpPerspective(denoised_image, transform_matrix, im.shape)
        # Add the transformed image to the list
        transformed_images.append(transformed_image)

    # Stack images and compute the median across the stack for each pixel
    transformed_stack = np.stack(transformed_images, axis=0)
    clean_baby_image = np.median(transformed_stack, axis=0).astype(np.uint8)

    return clean_baby_image
```

The result:



Windmill image:

In this image, we can see that there's a diagonal noise that is bold in the image, this means that in the frequency domain, the frequency that corresponds to this type of noise has a much larger coefficient than other frequencies in the frequency domain. Hence, we want to find this peak and set it to 0. To do so, we transformed the image to the frequency domain. Then, we excluded the window of size 5×5 around the center of the frequency domain (since, the closer a frequency to the center the lower it gets, and we search for the highest) by setting the window to false in the mask then later giving the window value of $-\infty$ to not be considered.

After that, we used np.argpartition(masked_spectrum.ravel(), -2)[-2:] to get the indices of the largest values based on the magnitude. These indices are then converted back to 2D coordinates using np.unravel_index.

Then we sorted these values in descending order and returned the first two (which are the highest).

After we found the peaks, we sat their value to 0 as intended and returned the image to spatial domain.

The code that finds peaks:

```python
def find_peaks(image):
    # Compute the Fourier Transform and shift the zero frequency component to the center
    fft_result = np.fft.fft2(image)
    shifted_fft = np.fft.fftshift(fft_result)

    # Compute the magnitude spectrum and apply logarithmic scaling
    magnitude_spectrum = np.log1p(np.abs(shifted_fft)) * 20

    # Create a boolean mask to exclude regions near the center
    center_x, center_y = image.shape[0] // 2, image.shape[1] // 2
    mask = np.ones_like(magnitude_spectrum, dtype=bool)
    mask[center_x - 2:center_x + 3, center_y - 2:center_y + 3] = False

    # Apply the mask to filter out the center
    masked_spectrum = np.where(mask, magnitude_spectrum, -np.inf)

    # Find the top two peak indices based on the magnitude
    peak_indices = np.argpartition(masked_spectrum.ravel(), -2)[-2:]
    peak_coordinates = np.array(np.unravel_index(peak_indices, magnitude_spectrum.shape)).T

    # Sort peaks by magnitude in descending order
    sorted_peaks = sorted(
        peak_coordinates, key=lambda coord: magnitude_spectrum[tuple(coord)], reverse=True
    )

    peaks = sorted_peaks[:2]
    peaks_coor = []
    for point in peaks:
        x, y = point
        peaks_coor.append((x, y))
    # Return as a list of tuples
    return peaks_coor
```

The code that cleans the image:

```python
def clean_windmill(im):
    # Compute the Fourier Transform and shift the zero frequency component to the center
    shifted_fourier = np.fft.fftshift(np.fft.fft2(im))

    # Define the coordinates of the peaks to be removed
    peaks = find_peaks(im)

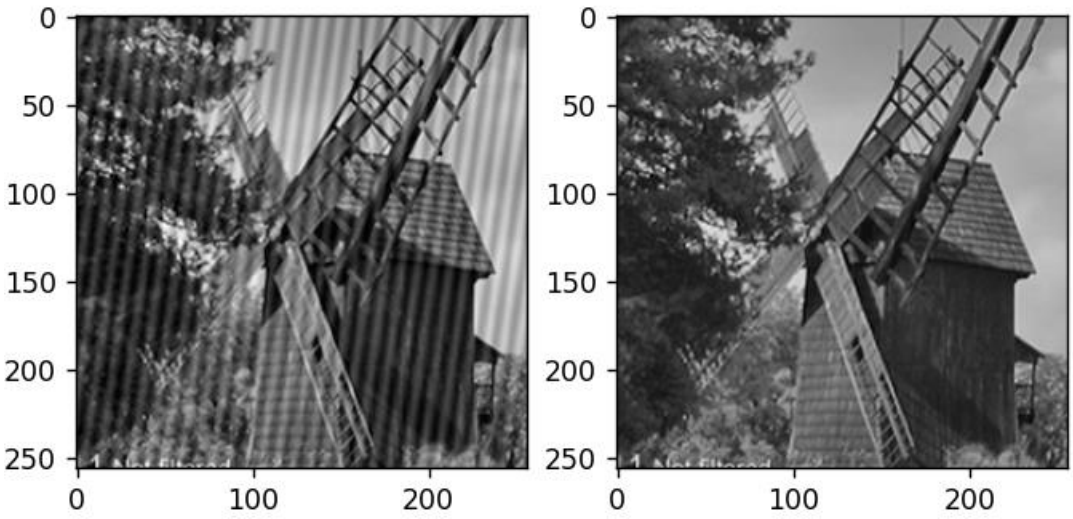
    # Use slicing and masking to remove peaks
    mask = np.ones_like(shifted_fourier, dtype=bool)
    for x, y in peaks:
        mask[x, y] = False

    # Apply the mask to the Fourier spectrum
    cleaned_spectrum = shifted_fourier * mask

    # Perform the inverse Fourier Transform to reconstruct the image
    cleaned_image = np.abs(np.fft.ifft2(np.fft.ifftshift(cleaned_spectrum)))

    return cleaned_image
```

The result:



Watermelon image:

This is a blurry image, we want to sharpen it. To do so, we created a sharpening high pass mask, then we applied the mask to the image and added the blurry image with the sharp one.

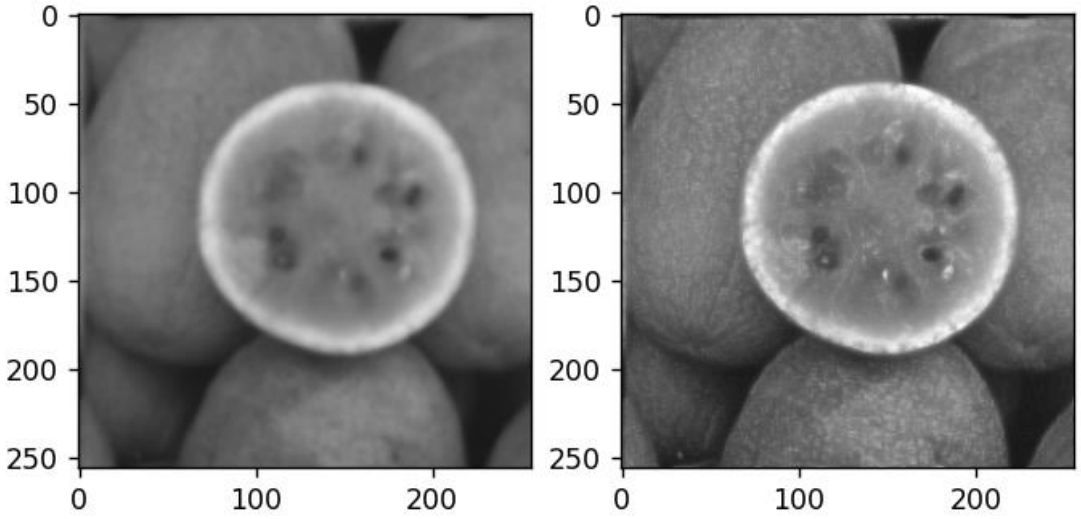
The code:

```
def clean_watermelon(im):
    kernel = np.array( object: [[0, -5, 0],
                        [-5, 20, -5],
                        [0, -5, 0]], dtype=np.float32)

    # Apply the high-pass filter
    filtered_img = cv2.filter2D(im, -1, kernel)

    # Add the filtered image to the original to sharpen it
    return cv2.add(im, filtered_img)
```

Result:



Umbrella image:

Given that the noised image I_n is the average of the original image I_o and shifted one I_s. Mathematically,

$$I_n = \frac{I_o + I_s}{2}$$

We can define I_s as a function of I_o in the following way:

Since I_s is shifted image of the original we can write it as

$$I_s(x, y) = I_o(x - x_0, y - y_0)$$

Where x_0, y_0 are the shifted values.

We want to obtain the original I_o, hence we'll isolate it:

$$2I_n = I_o + I_o(x - x_0, y - y_0)$$

To solve this problem, we'll apply fourier transform on both sides of the equation.

$$F\{2I_n\} = F\{I_o + I_o(x - x_0, y - y_0)\}$$

By the linearity property,

$$2F\{I_n\} = F\{I_o\} + F\{I_0(x - x_0, y - y_0)\}$$

Let's take a look at the shifted part:

$$F\{I_0(x - x_0, y - y_0)\} = F\{I_0\} \cdot e^{-2i\pi(ux_0 + vy_0)}$$

Where u, v are the axis in the frequency domain.

Hence,

$$2F\{I_n\} = F\{I_o\} + F\{I_0\} \cdot e^{-2i\pi(ux_0 + vy_0)} = F\{I_o\}\left(1 + e^{-2i\pi(ux_0 + vy_0)}\right)$$

In the end,

$$F\{I_o\} = \frac{2}{1 + e^{-2i\pi(ux_0 + vy_0)}} F\{I_n\}$$

Now, we can apply the inverse transform to return to the spatial domain to obtain the original image.

And that's what we did in the code.

Note: in the code, instead of dividing by numbers close to 0, we divided by 1 to avoid overfloating or dividing by 0.

Code:

```python
def clean_umbrella(im):
    # Get the image's dimension
    h, w = im.shape

    # Compute the Fourier Transform of the umbrella image
    fft_img = np.fft.fft2(im)

    # Define the shift
    x0, y0 = 79, 4

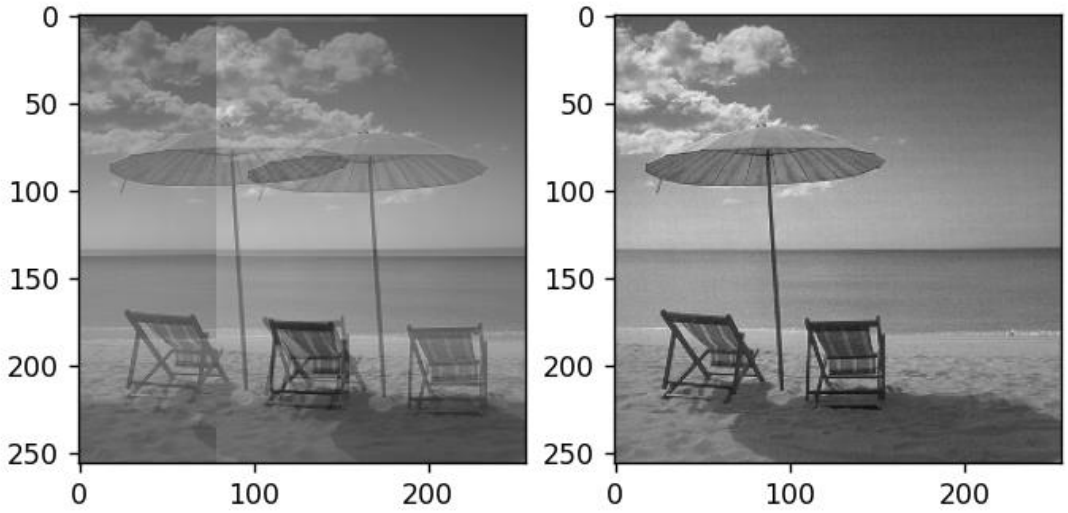
    # Extract frequencies in the Fourier domain
    v_axis = np.fft.fftfreq(h)
    u_axis = np.fft.fftfreq(w)
    [u, v] = np.meshgrid( *xi: u_axis, v_axis)

    # Compute the correction factor
    # How we got to this result is explained in the pdf
    correction_factor = 1 + np.exp(-2j * np.pi * (u * x0 + v * y0))

    # To avoid dividing by small number, we set the value to 1
    correction_factor = np.where(np.abs(correction_factor) < 0.01, 1, correction_factor)
    fft_img = 2 * fft_img / correction_factor

    # Transform back to the spatial domain
    return np.abs(np.fft.ifft2(fft_img))
```

Result:



USA flag:

In this image, we define the preserved regions (which are the regions that are clean and we want to them to be in the returned image), then we cleaned the image using horizontal average filter (since the strips are horizontal), after that we restored the preserved regions as before.

Code:

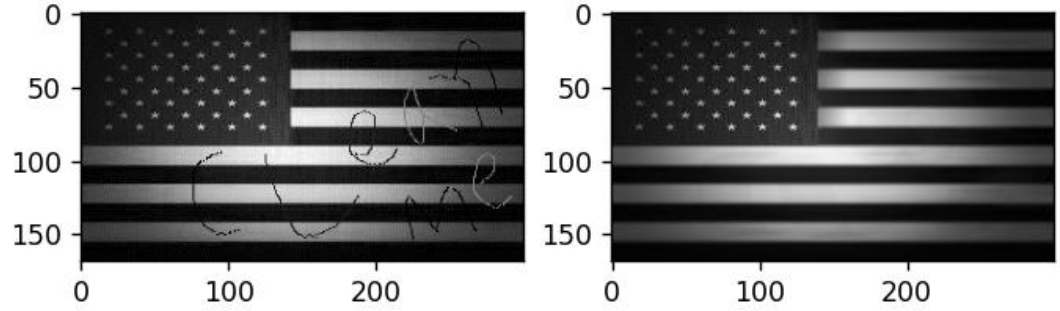
```python
def clean_USAflag(im):
    # Define the region to be preserved (top-left corner)
    region_height, region_width = 90, 140
    preserved_region = im[:region_height, :region_width]

    # Create a filtered version of the image (uniform filter along the x-axis)
    kernel = np.ones( shape: (3, 37), np.float32) / (3 * 37)
    filtered_image = cv2.filter2D(im, -1, kernel)

    # Replace the top-left region of the filtered image with the original region
    filtered_image[:region_height, :region_width] = preserved_region

    return filtered_image
```

Result:



House image:

As we can see, the images is horizontally blurry. To fix it, we need to transform to the frequency domain and modify the horizontal axis. So, we transformed to the frequency domain, then we defined the mask which is a horizontal line with small values (0.1- we choose this value because there are 10 images). Afterwards, we will undo the blur by dividing over the mask. Then return to the spatial domain.

Note: in the code, instead of dividing by numbers close to 0, we divided by 1 to avoid overfloating or dividing by 0.

Code:

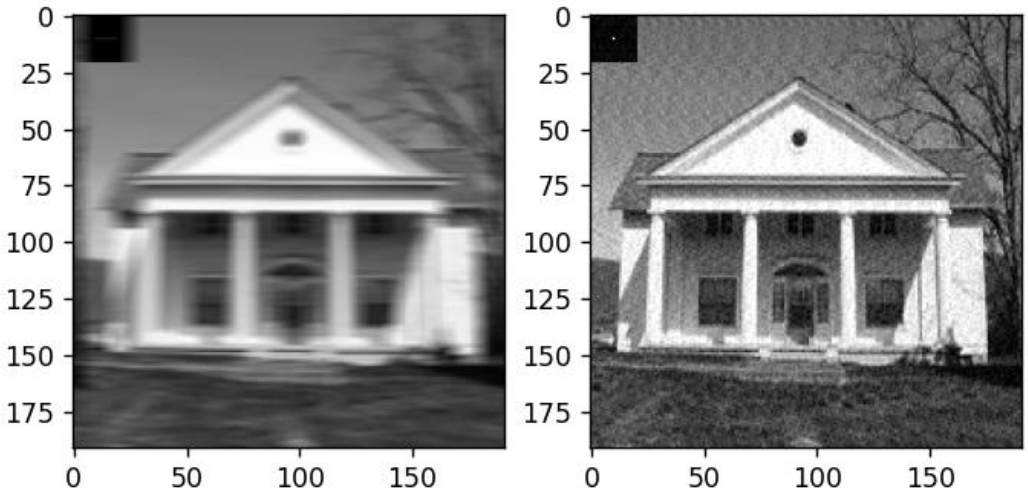
```
def clean_house(im):
    img_fourier = np.fft.fft2(im)
    # kernel for horizontal motion blur
    blur_mask = np.zeros(im.shape)

    blur_mask[0, 0:10] = 0.1

    blur_mask = np.fft.fft2(blur_mask)
    blur_mask = np.where(np.abs(blur_mask) < 0.001, 1, blur_mask)

    img_fourier = img_fourier / blur_mask
    cleaned_img = abs(np.fft.ifft2(img_fourier))
    return cleaned_img
```

Result:



Bears image:

In this image the contrast is low, we want to enhance it. Hence, we use image equalization to do this task.

Code:

```
def clean_bears(im):
    return cv2.equalizeHist(im)
```

Result:

